

Refactoring Strategies

Refactoring Strategies

Methods with Many Arguments

- **Question to Ask**: Can you replace arguments with a dictionary or another instance instead?
- **Question to Ask**: Are some of these arguments related? Do you use them enough that creating a new class or struct makes sense?
- **Strategy**: This is known as dependency injection and is a useful tool.

Example

Original:

```
def process_data(user_id, user_name, user_email, user_age):  
    # processing logic
```

Refactored:

```
class UserInfo:  
    def __init__(self, user_id, user_name, user_email, user_age):  
        self.user_id = user_id  
        self.user_name = user_name  
        self.user_email = user_email  
        self.user_age = user_age  
  
def process_data(user_info: UserInfo):  
    # processing logic
```

If Statements with Many Parameters

- **Strategy**: Try combining some conditions into another variable by giving this variable a meaningful name to make it more readable when used as part of a condition.

Example

Original:

```
if user.is_authenticated and user.has_permission("edit") and (user.role == "admin" or user.role == "editor") and not user.is_suspended and content.status in ["draft", "pending"]:
    content.publish()
```

Refactored:

```
# Define meaningful condition variables
is_user_authenticated: bool = user.is_authenticated
has_edit_permission: bool = user.has_permission("edit")
is_admin_or_editor: bool = user.role in ["admin", "editor"]
is_not_suspended: bool = not user.is_suspended
is_content_draft_or_pending: bool = content.status in ["draft", "pending"]

# Use the condition variables in the if statement
if (is_user_authenticated and has_edit_permission and is_admin_or_editor and is_not_suspended and is_content_draft_or_pending):
    content.publish()
```

Evaluating Tests

- **Strategy**: Make sure you don't have too many asserts in a single test. If so, it might make sense to break into multiple tests.

Example

Original:

```
def test_user_permissions():
    assert user.can_edit()
    assert user.can_view()
    assert user.can_delete()
```

Refactored:

```
def test_user_can_edit():
    assert user.can_edit()
```

```
def test_user_can_view():
    assert user.can_view()

def test_user_can_delete():
    assert user.can_delete()
```

Eliminating Reversed Processing

- **Strategy**: Look for any processing that gets done and then effectively reversed. Can this be eliminated or slightly modified to remove a lot of work?

Example

Original:

```
data = preprocess_data(raw_data)
processed_data = reverse_preprocess_data(data)
```

Refactored:

```
data = preprocess_data(raw_data, reverse=False)
```

Sharing Information Across Code

- **Strategy**: Create a file with dictionaries or structs that contains this info and import it to the relevant files, making it easy to change and modify when needed.

Example

Original:

```
def process_user(user_id):
    if user_id in [1, 2, 3]:
        # processing logic
```

Refactored:

```
# shared_info.py
USER_IDS = [1, 2, 3]

# main.py
from shared_info import USER_IDS

def process_user(user_id):
    if user_id in USER_IDS:
        # processing logic
```

Type Hinting Methods

- ****Strategy****: Type hint all your methods. If there is any type hint that is particularly wordy and difficult to read, use that as a hint to create new classes to pass instead.

Example

Original:

```
def calculate_area(length: float, width: float) -> float:
    return length * width
```

Refactored:

```
class Dimensions:
    def __init__(self, length: float, width: float):
        self.length = length
        self.width = width

def calculate_area(dimensions: Dimensions) -> float:
    return dimensions.length * dimensions.width
```

Reducing Logical Complexity

- ****Strategy****: When evaluating logical statements, see if any logical equivalencies can reduce the complexity of your logic. Replace if/for nests of more than three deep with function calls.

Example

Original:

```
if x > 0 and x < 10:
    if y > 0 and y < 10:
        if z > 0 and z < 10:
            # logic
```

Refactored:

```
def is_within_bounds(value, lower, upper):
    return lower < value < upper

if is_within_bounds(x, 0, 10) and is_within_bounds(y, 0, 10) and is_within_bounds(z, 0, 10):
    # logic
```

Method and Class Length

- **Strategy**: Methods longer than 20 lines or classes longer than 150 lines should be broken up into sub-methods and classes.

Example

Original:

```
def long_method():
    # 30 lines of code
```

Refactored:

```
def sub_method1():
    # 10 lines of code

def sub_method2():
    # 10 lines of code

def sub_method3():
    # 10 lines of code

def long_method():
    sub_method1()
    sub_method2()
    sub_method3()
```

Replace Array/Dict with Object

- **Strategy**: Replace arrays/dictionaries with various types of data with an object, and have each type of data reference a field in the object.

Example

Original:

```
data = {  
    "name": "John",  
    "age": 30,  
    "email": "john@example.com"  
}
```

Refactored:

```
class Person:  
    def __init__(self, name, age, email):  
        self.name = name  
        self.age = age  
        self.email = email  
  
data = Person("John", 30, "john@example.com")
```

Consolidate Similar Conditions

- **Strategy**: If you have multiple conditions that give similar or the same result, consolidate these into one condition using a method to prevent the condition from getting too large.

Example

Original:

```
if condition1 or condition2 or condition3:  
    perform_action()
```

Refactored:

```
def any_condition_met(*conditions):
    return any(conditions)

if any_condition_met(condition1, condition2, condition3):
    perform_action()
```

Removing Fragments from Conditions

- **Strategy**: If all your conditional statements have some fragment, that fragment can be removed from the condition.

Example

Original:

```
if age > 10:
    print(age)
    print("too old")
else:
    print(age)
    print("LETTTSSS G000!")
```

Refactored:

```
print(age)
if age > 10:
    print("too old")
else:
    print("LETTTSSS G000!")
```

Reducing Nested If Statements

- **Strategy**: When many nested if statements, we can often isolate special cases into guard clauses to reduce nesting.

Example

Original:

```
if x > 0:
    if y > 0:
        if z > 0:
            # logic
```

Refactored:

```
if x <= 0:
    return
if y <= 0:
    return
if z <= 0:
    return
# logic
```

Using Polymorphism

- **Strategy**: Use polymorphism to prevent constant type checking, allowing you to check parent types for behavior instead of dozens of different things, or custom write behavior of methods for each type to avoid conditionals.

Example

Original:

```
if isinstance(shape, Circle):
    shape.draw_circle()
elif isinstance(shape, Square):
    shape.draw_square()
```

Refactored:

```
class Shape:
    def draw(self):
        raise NotImplementedError

class Circle(Shape):
    def draw(self):
        # draw circle

class Square(Shape):
    def draw(self):
```

```
# draw square

def draw_shape(shape: Shape):
    shape.draw()
```

Assertions for State-Specific Code

- **Strategy**: If code needs to be in a specific state for some code to run, introduce assertion statements that can serve as living documentation and help with debugging.

Example

Original:

```
# assume certain state
process_data()
```

Refactored:

```
assert state == "expected_state", "State must be 'expected_state' to run this code"
process_data()
```

Splitting Methods That Modify and Return a Value

- **Strategy**: Split a method into two—one to modify and one to return if the method modifies and returns a value.

Example

Original:

```
def modify_and_return(value):
    value += 10
    return value
```

Refactored:

```
def modify_value(value):  
    return value + 10  
  
def return_value(value):  
    return value
```

Generalizing Similar Methods

- **Strategy**: If methods do similar things, generalize these methods into a single method.

Example

Original:

```
def process_user_data(user):  
    # process user  
  
def process_admin_data(admin):  
    # process admin
```

Refactored:

```
def process_data(entity):  
    # process entity
```